

- Description

This module manages a smart data center temperature monitoring and cooling optimization system, ensuring that:

1. **The temperature of server racks is continuously monitored to prevent overheating.**
2. **If the temperature exceeds a predefined threshold, cooling systems are activated or intensified.**
3. **If the temperature drops below the optimal efficiency range, cooling power is reduced to save energy.**
4. **A log of all temperature readings, cooling actions, and power optimizations is maintained for system reliability.**

This system helps prevent hardware failures, reduce energy consumption, and optimize data center cooling efficiency.

- Specification

module SmartDataCenterCooling

var currentTemperature, maxTemperature, optimalMinTemperature,
optimalMaxTemperature: real

var coolingActive: boolean

var log: sequence of string

process updateTemperature(changeAmount: real)

functional scenario form:

currentTemperature <= maxTemperature && log = log $\oplus$ ("Temperature is Optimal: " + currentTemperature + "°C."), Temperature updated successfully.

||

currentTemperature > maxTemperature && coolingActive = true, log = log $\oplus$ ("Overheating Warning: " + currentTemperature + "°C, Increasing Cooling Power."), Overheating warning triggered, cooling increased.

||

currentTemperature < optimalMinTemperature && coolingActive = false, log = log $\oplus$ ("Low Temperature Alert: " + currentTemperature + "°C, Reducing Cooling for Energy Saving."), Low temperature alert triggered, cooling reduced.

end process

end module

- Java Code

```
package code;
import java.util.*;
class DataCenterCooling {
    private double maxTemperature;
    private double optimalMinTemperature;
    private double optimalMaxTemperature;
    private double currentTemperature;
```

```

    private boolean coolingActive;
    private List<String> log;
    public DataCenterCooling(double maxTemperature, double optimalMinTemperature, double optimalMaxTemperature, double initialTemperature) {
        this.maxTemperature = maxTemperature;
        this.optimalMinTemperature = optimalMinTemperature;
        this.optimalMaxTemperature = optimalMaxTemperature;
        this.currentTemperature = initialTemperature;
        this.coolingActive = false;
        this.log = new ArrayList<>();
    }
    public void updateTemperature(double changeAmount) {
        currentTemperature += changeAmount;
        manageCooling();
        String message = String.format("Current Temperature: %.2f°C", currentTemperature);
        if (currentTemperature > maxTemperature) {
            message += " | WARNING: Overheating Risk! Increasing Cooling Power.";
        } else if (currentTemperature < optimalMinTemperature) {
            message += " | ALERT: Low Temperature! Reducing Cooling to Save Energy.";
        } else if (currentTemperature > optimalMaxTemperature) {
            message += " | INFO: Adjusting Cooling for Efficiency.";
        } else {
            message += " | Temperature is Optimal.";
        }
        System.out.println(message);
        log.add(message);
    }
    private void manageCooling() {
        if (currentTemperature > maxTemperature) {
            coolingActive = true;
        } else if (currentTemperature < optimalMinTemperature) {
            coolingActive = false;
        }
    }
    public void printLog() {
        System.out.println("\nData Center Temperature Log:");
        for (String logEntry : log) {
            System.out.println(logEntry);
        }
    }
}

```

```

public class c49_SmartDataCenterSystem {
    public static void main(String[] args) {
        DataCenterCooling dataCenter = new DataCenterCooling(35.0, 18.0
, 28.0, 25.0);
        // testcase-VT:
        //     dataCenter.updateTemperature(4.0);
        //     dataCenter.updateTemperature(-6.0);
        //     dataCenter.updateTemperature(-8.0);
        //     dataCenter.updateTemperature(12.0);
        //     dataCenter.updateTemperature(10.0);
        //     dataCenter.printLog();

        // testcase-FT:
        //     dataCenter.updateTemperature(4.9E-324);
        //     dataCenter.updateTemperature(-1.7976931348623157E308);
        //     dataCenter.updateTemperature(0.0);
        //     dataCenter.updateTemperature(6.750504225312172E302);
        //     dataCenter.updateTemperature(-1.4170051768914723E308);
        //     dataCenter.printLog();

        // testcase-Z3:
        //     dataCenter.updateTemperature(4);
        //     dataCenter.updateTemperature(-6);
        //     dataCenter.updateTemperature(-8);
        //     dataCenter.updateTemperature(12);
        //     dataCenter.updateTemperature(10);
        //     dataCenter.printLog();

        // testcase-UVT:
        dataCenter.updateTemperature(4.0);
        dataCenter.updateTemperature(-6.0);
        dataCenter.updateTemperature(-8.0);
        dataCenter.updateTemperature(12.0);
        dataCenter.updateTemperature(10.0);
        dataCenter.printLog();
    }
}

```

- Coverage report

1. Test case number: 5
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
√ VibrationTestingTool	1.4 %	12	830	842
√ src	1.4 %	12	830	842
√ code	1.4 %	12	830	842
√ c49_SmartDataCenterSystem.java	100.0 %	12	0	12
> c49_SmartDataCenterSystem		0	0	0
> DataCenterCooling	100.0 %	12	0	12

3. Vibration testing instruction coverage: 98%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
√ VibrationTestingTool	0.7 %	149	21,318	21,467
√ src	0.7 %	149	21,318	21,467
√ code	0.7 %	149	21,318	21,467
√ c49_SmartDataCenterSystem.java	98.0 %	149	3	152
> c49_SmartDataCenterSystem	89.7 %	26	3	29
> DataCenterCooling	100.0 %	123	0	123

4. Fuzz testing branch coverage: 75%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
√ VibrationTestingTool	1.1 %	9	833	842
√ src	1.1 %	9	833	842
√ code	1.1 %	9	833	842
√ c49_SmartDataCenterSystem.java	75.0 %	9	3	12
> c49_SmartDataCenterSystem		0	0	0
> DataCenterCooling	75.0 %	9	3	12

5. Fuzz testing instruction coverage: 90.1%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
√ VibrationTestingTool	0.6 %	137	21,330	21,467
√ src	0.6 %	137	21,330	21,467
√ code	0.6 %	137	21,330	21,467
√ c49_SmartDataCenterSystem.java	90.1 %	137	15	152
> c49_SmartDataCenterSystem	89.7 %	26	3	29
> DataCenterCooling	90.2 %	111	12	123

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
√ VibrationTestingTool	1.4 %	12	830	842
√ src	1.4 %	12	830	842
√ code	1.4 %	12	830	842
√ c49_SmartDataCenterSystem.java	100.0 %	12	0	12
> c49_SmartDataCenterSystem		0	0	0
> DataCenterCooling	100.0 %	12	0	12

7. Z3 instruction coverage: 98%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
√ VibrationTestingTool	0.7 %	149	21,318	21,467
√ src	0.7 %	149	21,318	21,467
√ code	0.7 %	149	21,318	21,467
√ c49_SmartDataCenterSystem.java	98.0 %	149	3	152
> c49_SmartDataCenterSystem	89.7 %	26	3	29
> DataCenterCooling	100.0 %	123	0	123

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
√ VibrationTestingTool	1.4 %	12	830	842
√ src	1.4 %	12	830	842
√ code	1.4 %	12	830	842
√ c49_SmartDataCenterSystem.java	100.0 %	12	0	12
> c49_SmartDataCenterSystem		0	0	0
> DataCenterCooling	100.0 %	12	0	12

9. Unimproved V-method instruction coverage: 98%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
VibrationTestingTool	0.7 %	149	21,318	21,467
src	0.7 %	149	21,318	21,467
code	0.7 %	149	21,318	21,467
c49_SmartDataCenterSystem.java	98.0 %	149	3	152
c49_SmartDataCenterSystem	89.7 %	26	3	29
DataCenterCooling	100.0 %	123	0	123